Packaging Subtitles

Unified Packager allows you to package and prepare your subtitles for streaming delivery (using statically packaged files, or dynamic packaging with Origin):

- General workflow for adding subtitles to a stream
- Supported formats for subtitles
  - Supported TTML profiles
  - Difference between WebVTT and SRT
- Packaging TTML, WebVTT or SRT in fMP4
  - WebVTT (or SRT) in fMP4
  - TTML in fMP4
- Converting WebVTT (or SRT) to TTML
  - Supported cue components
- Converting TTML to WebVTT
- Extracting embedded captions (to TTML or WebVTT)

General workflow for adding subtitles to a stream

Whether you are preparing your subtitles for streaming delivery using static packaging with Packager or dynamic packaging with Origin, the general rule is that subtitles have to be packaged in a fMP4 container (`.ismt` or `.cmft`) before they can be added to a stream. All styling information and editorial changes should be made before packaging using the relevant encoder or subtitle tooling. When packaged in a fMP4 container, adding a subtitle track to a stream works the same as adding audio or video tracks:

- For streaming delivery using statically packaged files, add the `.ismt` or `.cmft` with subtitles to your `mp4split` input when generating the client manifest (`.mpd` or `.m3u8`)
- For streaming delivery using dynamic packaging with Origin for VOD, add the `.ismt` or `.cmft` with subtitles to your `mp4split` input when generating the server manifest (`.ism`)
- For streaming delivery using dynamic packaging with Origin for Live, the encoder should POST the subtitles one track per language to the publishing point

How you should package your subtitles in a fMP4 container is explained on this page, where you are now. Example command-lines for adding fMP4 packaged subtitles to different kinds of streams can be found in the relevant parts of the documentation, listed below:

- How to add subtitles to statically packaged Apple HLS TS
- How to add subtitles to statically packaged MPEG-DASH
- How to add subtitles to a VOD stream using Origin for VOD
- How to add subtitles to a livestream using Origin for Live

Note

The three exceptions to the general rule that you need to package your subtitles in a fMP4 container before you can add them to a stream are:

- Signaling embedded 608/708 Closed Captions (protocol version 4)
- Adding TTML or WebVTT sidecar subtitles for MPEG-DASH
- Embedded CEA-608 or CEA-708 captions when streaming Live

Supported formats for subtitles

You can use TTML (Timed Text Markup Language), WebVTT (Web Video Text Tracks) or SRT (SubRip Text) as your source and use Packager to convert one to the other, as well as package TTML and WebVTT in a fMP4 container.

For more information on these different formats, please read our blog about subtitles: Welcome to the jungle: caption and subtitle formats in video streaming. In short, WebVTT and SRT are nearly identical formats in plain-text, whereas TTML is XML-based.

New in version 1.10.16.

In addition to the above it is possible to extract subtitles from a CEA-608 embedded captions track, and store them as TTML or WebVTT.

Source	Possible outputs
TTML	WebVTT, TTML in fMP4
WebVTT (or SRT)	TTML, WebVTT in fMP4
CEA-608	TTML, WebVTT

Supported TTML profiles

The TTML specification defines the use of profiles. Each profile specifies a certain feature set. You can learn more about these profiles and their features in our blog about subtitles: Welcome to the jungle: caption and subtitle formats in video streaming. Packager can package TTML

subtitles that follow any of the following profiles: DFXP, SMPTE-TT, EBU-TT-D, SDP-US, CFF-TT and the IMSC1 Text Profile. *Unified Origin* supports all of those profiles as well.

Difference between WebVTT and SRT

WebVTT is based on SRT and both are very similar, with only small differences in formatting. Overall, the most important difference is that the WebVTT has an official specification that is recommended by W3C and that allows for more advanced formatting features (such as positioning).

When using WebVTT or SRT as input for `mp4split`, do consider that:

- For SRT, `mp4split` assumes the input file is encoded in ASCII unless it starts with a Byte Order Marker (BOM) that describes how the input should be transformed to Unicode
- For WebVTT, `mp4split` always interprets the input files as being encoded as Unicode (regardless of any BOM), because WebVTT is UTF-8 by definition

> **Note**
>
> Both WebVTT and SRT do not contain signaling for the language of the subtitles in the file. Therefore, always specify the language when using WebVTT or SRT as input for `mp4split` (using the `--track_language` command-line option). Otherwise, the language that is signaled defaults to English.

Packaging TTML, WebVTT or SRT in fMP4

When you use Packager to package your subtitles in a fMP4 container, we follow ISO 14496-30. This results in the following:

- When using WebVTT (or SRT) as input, the resulting fMP4 will use the `wvtt` codec
- When using TTML as input, the resulting fMP4 will use the `stpp` codec

There are only two exceptions to this rule, which are related to packaging TTML and explained in the relevant section below.

When packaging subtitles in an fMP4 container, the following options may be relevant:

- --track_language: When you need to add (for WebVTT or SRT) or overrule language signaling. (If the source does not contain language signaling and you do not add any, English is the default.)

- --track_role and --track_kind: When you need to define a 'role' for the subtitles track, or want to add signaling for an accessibility feature.
- --fragment_duration: When you want to specify the duration of the fMP4 media fragments in which the subtitles are chunked. E.g. to align it with the fragments (GOPs) of the other media in your presentation. (The default for subtitles is to create a sample for each separate subtitle cue (progressive) or 2s fragments (cmft/ismt).)

WebVTT (or SRT) in fMP4

New in version 1.7.31.

To create a fMP4 with subtitles that are formatted according to the `wvtt` codec, use WebVTT (or SRT) subtitles as input. To avoid confusion about character encoding, we recommend WebVTT which uses UTF-8 by definition. You should always specify the language of the track that you are packaging (using --track_language), because WebVTT and SRT files do not contain language signaling. We recommend using a fragment duration that aligns with the other tracks in the presentation (using --fragment_duration):

```
#!/bin/bash

mp4split -o tears-of-steel-wvtt-nl.ismt \
  --fragment_duration=60/1 \
  tears-of-steel-nl.webvtt --track_language=nl
mp4split -o tears-of-steel-wvtt-de.ismt \
  --fragment_duration=60/1 \
  tears-of-steel-de.srt --track_language=de
```

Note

Packaging `wvtt` allows for WebVTT specific *cue settings* to define individual subtitle positioning, region and styling information. This provides more detailed control over WebVTT than relying on *Unified Origin* to transcode WebVTT fragments from TTML formatted subtitles.

TTML in fMP4

To create a fMP4 with subtitles that are formatted according to the `stpp` codec, use TTML subtitles as input: [1]

```bash
#!/bin/bash

mp4split -o tears-of-steel-ttml-nl.ismt \
  tears-of-steel-nl.ttml
```

This command creates a file with a single track, which is why the TTML input file should contain only one language. If you have a single TTML file that contains multiple languages then you will have to extract separate TTML files for each language first.

As already noted above, there are two exceptions to take into account when packaging TTML in fMP4:

- When you use SMPTE-TT formatted TTML with bitmaps as your input, the samples in the fMP4 are automatically formatted according to SMPTE-TT specification
- When you are statically packaging HTTP Smooth Streaming (Packaging for HTTP Smooth Streaming (HSS)), you should use command-line option `--brand=piff` to ensure that the older `dfxp` codec is used, so that the timing of the `@begin` and `@end` attributes in the resulting fMP4 is relative to the start of each sample, instead of relative to the start of the track

> **Note**
>
> The distinction between the `stpp` and `dfxp` codec is only relevant for statically packaged content. When you are working with *Unified Origin*, timing will be adjusted automatically if necessary.

Converting WebVTT (or SRT) to TTML

When you convert WebVTT or SRT to TTML, the TTML will have a default styling and layout that in general should work well (see the overview of supported cue components below). To convert WebVTT or SRT to TTML, use a WebVTT or SRT file as input and specify an output with `.ttml` or `.dfxp` as the extension.

```bash
#!/bin/bash

mp4split -o tears-of-steel-nl.ttml \
  tears-of-steel-nl.webvtt --track_language="nl"

mp4split -o tears-of-steel-fr.ttml \
  tears-of-steel-fr.srt --track_language="fr"
```

Supported cue components

When converting WebVTT or SRT to TTML, only a limited set of markup features is converted to their TTML equivalents. Others are either ignored or escaped (see the example below). The markup features that will be converted are the following:

Name	Description
`<b></b>`	Bolds the textual content
`<i></i>`	Italicises the text
`<u></u>`	Underlines the textual content
`<s></s>`	Specifies a line strike through on the text

Here is an example of a regular WebVTT file with some cue point component elements:

WebVTT cue point example :

```
WEBVTT

1
00:00:15,000 --> 00:00:18.000
At the <u>left</u> we can see...

2
00:00:18,167 --> 00:00:20,083 position:35% line:20 align:left
At the <u>right</u> we can see the...

3
00:00:20,083 --> 00:00:22.000
...the <c.highlight>head-snarlers</c>

4
00:00:22,000 --> 00:00:24.417
Everything is safe.
<i>Perfectly</i> safe.
```

Result after converting to TTML:

```
<?xml version="1.0" encoding="utf-8"?>
<tt xmlns="..." xml:lang="en">
  <head>...</head>
  <body>
    <div xml:lang="en">
      <p begin="00:00:15.000" end="00:00:18.000" region="speaker">
        At the <span tts:textDecoration="underline">left</span> we can see...
      </p>
      <p begin="00:00:18.167" end="00:00:20.083" region="speaker">
        At the <span tts:textDecoration="underline">right</span> we can see the...
      </p>
      <p begin="00:00:20.083" end="00:00:22.000" region="speaker">
        ...the &lt;c.highlight&gt;head-snarlers&lt;/c&gt;
      </p>
      <p begin="00:00:22.000" end="00:00:24.417" region="speaker">
        Everything is safe.<br />
        <span tts:fontStyle="italic">Perfectly</span> safe.
      </p>
    </div>
  </body>
</tt>
```

Note

The *settings* (cue 2) are ignored when converting to TTML and unrecognized styling in the payload is escaped (cue 3).

Footnote

[1] To create text samples it is important that *Unified Packager* can derive correct timing information from TTML source. While the TTML spec is liberal (and sometimes ambiguous) in this respect, Packager assumes timing in `HH:MM:SS.mmm` format in the `@begin` and `@end` attributes of `tt/body/div/p` element. Timing at a different element under `tt/body` is allowed, but only at the same level. For instance, SMPTE-TT encoders may choose either `tt/body/div` or `tt/body/div/div` but should not use both in one file.

Converting TTML to WebVTT

In general, TTML offers a lot more flexibility regarding document structure and styling of cues. When converting TTML to WebVTT, only a subset of this extra information will be maintained:

- Bold text
- Italicized text
- Underlined text
- Strike through text

Also, only explicit line breaks will be respected (`<br />`), meaning cues spread out over more than one paragraph (`<p>`) will end up on one line in WebVTT.

> **Note**
>
> Converting image-based TTML to WebVTT is not supported. When using image-based TTML as an input for Origin, use Using dynamic track selection to filter out the image-based TTML input when requesting HLS.

Extracting embedded captions (to TTML or WebVTT)

To extract embedded captions from a video track, specify the video track carrying the embedded captions as input and specify an output with either a `.webvtt` or `.ttml` extension, depending on the format in which you want to store the extracted captions:

```
#!/bin/bash

mp4split -o captions.ttml \
  video-with-captions.mp4 --track_type=video

mp4split -o captions.webvtt \
  video-with-captions.mp4 --track_type=video
```

When extracting the captions, Packager will take the *language* attribute from the video track that carries the embedded captions. When present, this attribute is added to TTML output (for WebVTT it will not, because it does not support language signaling). To make Packager specify a different language in its TTML output, use its `--track_language` option:

```
#!/bin/bash

mp4split -o captions.ttml \
  video-with-captions.mp4 --track_type=video --track_language="sp"
```